

NAME: Atchaya Vharsne S

REG. NO.: 192524185

COURSE CODE: CSA0401

COURSE NAME: OPERATING SYSTEMS

CO5 AT1 CONSTRAINT BASED PROBLEM SOLVING

1. Optimized Linux File System Layout for a 50 GB Server

A proper partition layout prevents logs or user files from filling the entire disk and affecting critical services.

Suggested disk layout

Partition	Size	Purpose
/boot	1 GB	Boot files and kernel
/	12 GB	Operating system and applications
/var	10 GB	Logs, cache, spool files
/home	12 GB	User files
/tmp	2 GB	Temporary files
Swap	2 GB	Virtual memory
Free/LVM space	11 GB	Future expansion

Storage management techniques

- Use **LVM (Logical Volume Manager)** for flexible partition resizing.
- Keep /var separate so excessive log generation does not fill the root partition.
- Configure **logrotate** to rotate, compress, and remove old log files.

- Set log retention policies so logs remain within the **10 GB limit**.
- Monitor disk usage using:
- `df -h`

`du -sh /var/log/*`

- Set alerts when disk usage reaches a threshold such as 80–90%.

Log rotation example

The `/etc/logrotate.conf` configuration can rotate logs daily or weekly, compress old logs, and retain only a specified number of copies.

Result: Separating critical directories, using LVM, and applying log rotation ensures that logs do not exceed 10 GB and prevents disk exhaustion from interrupting important services.

2. Kernel-Level Optimization for High CPU Workloads

Frequent context switching occurs when the CPU repeatedly switches between processes or threads. This creates overhead and reduces performance.

Proposed techniques

1. Scheduler tuning

The Linux scheduler can be tuned to reduce unnecessary context switching.

Useful tools include:

`chrt`

`taskset`

`nice`

`renice`

- `nice` and `renice` adjust process priority.
- `taskset` binds a process to specific CPU cores using CPU affinity.
- `chrt` assigns real-time scheduling policies such as `SCHED_FIFO` or `SCHED_RR`.

Example:

```
taskset -c 0,1 ./application
```

This keeps the process on selected CPU cores and may reduce migration and context-switch overhead.

2. Use real-time scheduling

For time-critical tasks:

```
chrt -f 80 ./application
```

This assigns a FIFO real-time priority.

3. Optimize kernel modules

- Remove unnecessary modules using:

```
rmmod module_name
```

- Load only required modules:

```
modprobe module_name
```

- Reduce unnecessary interrupts and background kernel activity.

4. Monitor context switching

Use:

```
vmstat 1
```

```
pidstat -w
```

```
top
```

```
htop
```

Result: CPU affinity, priority tuning, real-time scheduling, and optimized kernel modules reduce scheduling overhead and improve performance without hardware upgrades.

3. Processing 1 Million Log Entries per Day Using Shell Scripting

To process a large number of log entries efficiently, optimized Linux commands should be used instead of slow line-by-line shell loops.

Efficient approach

```
awk '/ERROR/ {count++} END {print "Total Errors:", count}' logfile.log
```

For searching specific patterns:

```
grep -F "ERROR" logfile.log
```

For counting:

```
grep -Fc "ERROR" logfile.log
```

For parallel processing of multiple log files:

```
find /logs -type f -print0 | xargs -0 -P 4 grep -H "ERROR"
```

Optimized command choices

- **grep -F**: Faster for fixed string matching because it does not interpret regular expressions.
- **awk**: Processes files efficiently in a single pass and supports filtering and calculations.
- **sort**: Used for sorting large datasets.
- **uniq -c**: Efficiently counts repeated entries.
- **xargs -P**: Enables parallel processing.

Example pipeline:

```
grep -F "ERROR" logfile.log | awk '{print $NF}' | sort | uniq -c
```

Avoid commands such as repeatedly calling `cat`, `grep`, or `awk` inside loops because creating many processes increases execution time.

Result: Using optimized tools such as `grep -F`, `awk`, `sort`, `uniq`, and parallel processing allows millions of log entries to be processed efficiently within the required 2-minute limit.

4. Identifying and Controlling a Process Consuming 90% CPU

The faulty process can be identified without restarting the system.

Step 1: Identify the process

`top`

or:

`ps aux --sort=-%cpu | head`

This displays the processes consuming the highest CPU resources.

Step 2: Monitor the process

`top -p PID`

or:

`pidstat -p PID 1`

Step 3: Reduce its priority

`renice +10 -p PID`

A higher nice value lowers the CPU scheduling priority.

Step 4: Restrict the process to a CPU core

`taskset -cp 1 PID`

Step 5: Pause the process temporarily

`kill -STOP PID`

To continue:

`kill -CONT PID`

Step 6: Terminate only if necessary

`kill PID`

If it does not respond:

`kill -9 PID`

Result: Tools such as top, ps, pidstat, renice, taskset, and kill allow the administrator to identify, monitor, control, or terminate the faulty process while keeping the remaining critical services running.

5. Virtualization Strategy for Hosting 5 Virtual Machines

Given

- Total RAM = **16 GB**
- Host operating system requires = **4 GB**
- Remaining RAM = **12 GB**
- Each VM requires a minimum of = **2 GB**
- Total required for 5 VMs = **10 GB**

Therefore:

So, the system has:

Proposed allocation

System	RAM Allocation
Host OS	4 GB
VM 1	2 GB
VM 2	2 GB
VM 3	2 GB
VM 4	2 GB
VM 5	2 GB
Remaining	2 GB

Virtualization techniques

- Use a **Type-1 or efficient Type-2 hypervisor** depending on the deployment environment.
- Allocate a guaranteed **2 GB RAM** to each VM.
- Keep the remaining 2 GB as a buffer for the hypervisor and temporary workload increases.
- Use **dynamic memory allocation, ballooning, or memory overcommit** carefully if supported.
- Use lightweight guest operating systems where possible.
- Monitor memory and CPU usage continuously.

Conclusion: The 5 VMs require only 10 GB, while 12 GB is available after reserving 4 GB for the host OS. Therefore, all VMs can run smoothly with 2 GB each and a 2 GB reserve for system stability.
